

# Pick Word 🐍 🐍

## Exercise 30 (and [Solution](#))

*This exercise is Part 1 of 3 of the Hangman exercise series. The other exercises are: [Part 2](#) and [Part 3](#).*

In this exercise, the task is to write a function that picks a random word from a list of words from the [SOWPODS dictionary](#). Download this file and save it in the same directory as your Python code. This file is [Peter Norvig](#)'s compilation of the dictionary of words used in professional Scrabble tournaments. Each line in the file contains a single word.

*Hint: use the Python `random` library for picking a random word.*

### Aside: what is SOWPODS

SOWPODS is a word list commonly used in word puzzles and games (like Scrabble for example). It is the combination of the Scrabble Player's Dictionary and the Chamber's Dictionary. (The history of SOWPODS is quite interesting, I highly recommend reading the [Wikipedia article if you are curious](#).)

# Guess Letters 🐍🐍

## Exercise 31 (and [Solution](#))

*This exercise is Part 2 of 3 of the Hangman exercise series. The other exercises are: [Part 1](#) and [Part 3](#).*

Let's continue building Hangman. In the game of Hangman, a clue word is given by the program that the player has to guess, letter by letter. The player guesses one letter at a time until the entire word has been guessed. (In the actual game, the player can only guess 6 letters incorrectly before losing).

Let's say the word the player has to guess is "EVAPORATE". For this exercise, write the logic that asks a player to guess a letter and displays letters in the clue word that were guessed correctly. For now, let the player guess an infinite number of times until they get the entire word. As a bonus, keep track of the letters the player guessed and display a different message if the player tries to guess that letter again. Remember to stop the game when all the letters have been guessed correctly! Don't worry about choosing a word randomly or keeping track of the number of guesses the player has remaining - we will deal with those in a future exercise.

An example interaction can look like this:

```
>>> Welcome to Hangman!
_ _ _ _ _
>>> Guess your letter: S
Incorrect!
>>> Guess your letter: E
E _ _ _ _ _ E
...
```

And so on, until the player gets the word.

# Hangman 》》

## Exercise 32 (and [Solution](#))

*This exercise is Part 3 of 3 of the Hangman exercise series. The other exercises are: [Part 1](#) and [Part 2](#).*

In this exercise, we will finish building Hangman. In the game of Hangman, the player only has 6 incorrect guesses (head, body, 2 legs, and 2 arms) before they lose the game.

In Part 1, we loaded a random word list and picked a word from it. In Part 2, we wrote the logic for guessing the letter and displaying that information to the user. In this exercise, we have to put it all together and add logic for handling guesses.

Copy your code from Parts 1 and 2 into a new file as a starting point. Now add the following features:

- Only let the user guess 6 times, and tell the user how many guesses they have left.
- Keep track of the letters the user guessed. If the user guesses a letter they already guessed, don't penalize them - let them guess again.

Optional additions:

- When the player wins or loses, let them start a new game.
- Rather than telling the user "You have 4 incorrect guesses left", display some picture art for the Hangman. *This is challenging - do the other parts of the exercise first!*

Your solution will be a lot cleaner if you make use of functions to help you!